université
PARIS-SACLAY

RAPPORT DE TP

VISION ROBOTIQUE

Auteur :

Gaëtan BOUCHY
Thomas ABOU JAOUDE

Enseignant :

M. LERME

À Orsay, le 26/02/2022

université
PARIS-SACLAY

Sommaire

Sommaire 2
1. Détection du fond 3
1.1 Calcul de la moyenne et de l'écart-type et classification des pixels en fond / voiture ... 3
1.1.1 Calcul de la moyenne et de l'écart-type 3
1.1.2 Classification des pixels en fond / voiture 6
1.2 Analyse de la variation du niveau de gris du pixel (200,150). Identification des plateaux dans les niveaux de gris de pixels pour faire l'estimation de la moyenne et de l'écart-type 10
1.2.1 Analyse de la variation du niveau de gris du pixel (200,150) 10
1.2.2 Identification des plateaux dans les niveaux de gris de pixels pour faire l'estimation de la moyenne et de l'écart-type 11
1.3 Faites tourner un algorithme ΣΔ 18
1.4 Séparation lors de la détection des voitures et de leurs ombres 19
1.4.1 Avec l'espace YCbCr 19
1.4.2 Avec l'espace HSV fusionné à l'espace YCbCr 24
2. Flot optique 29
2.1 Introduction 29
2.2 Explication de l'algorithme 30
2.3 Utilité du paramètre α 31
3. Annexes 32
3.1 CodeMatlab 32

université PARIS-SACLAY

1. Détection du fond

1.1 Calcul de la moyenne et de l'écart-type et classification des pixels en fond / voiture

1.1.1 Calcul de la moyenne et de l'écart-type

Notre première étape a tout d'abord été de charger les images via **imread()** avec leur nom de façon correcte. Pour cela nous avons réalisé trois cas en fonction du nombre de 0 dans les images. Puis nous avons converti l'image chargée en niveau de gris :

```
while (compteurIteration < nbIterations)

    if(compteurIteration < 10)
        I = imread(['image_00' num2str(compteurIteration, '%01d') '.jpg']);
    end

    if(compteurIteration >= 10 && compteurIteration < 100)
        I = imread(['image_0' num2str(compteurIteration, '%02d') '.jpg']);
    end

    if(compteurIteration >= 100)
        I = imread(['image_' num2str(compteurIteration, '%02d') '.jpg']);
    end

    if ndims(I) == 3
        I = rgb2gray(I); % Convertir en niveaux de gris
    end
```

Figure 1 : Code permettant de charger et de convertir une image en niveaux de gris

Notre seconde étape a été de calculer la moyenne. Pour cela, nous devions sommer les valeurs d'un pixel par exemple le pixel(100,100) sur toutes les images puis nous devions diviser par le nombre d'images. Nous devions réaliser cela pour chaque pixel de chaque image. Cette étape a été réalisée dans une boucle while de la taille du nombre d'images :

```
for colonne = 1:hauteur
    for ligne = 1:largeur
        moyenne(colonne,ligne) = moyenne(colonne,ligne)+int32(I(colonne,ligne));
        valeurPixel(colonne,ligne,compteurIteration) = I(colonne,ligne);
    end
end
```

Figure 2 : Code permettant de charger et de convertir une image en niveaux de gris

Puis la troisième étape consistait à calculer l'écart-type. Pour cela, nous avons utilisé la formule de l'écart-type qui se présente comme suit :

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2}$$

Figure 3 : Formule de l'écart-type

Avec :
- σ est l'écart type de l'ensemble de données
- Σ correspond à la somme
- x_i représente chaque observation dans l'ensemble de données
- $\bar{x}$ est la moyenne de l'ensemble de données
- n est le nombre total d'observations dans l'ensemble de données

Ainsi, nous obtenons ce code qui permet de calculer l'écart-type :

```
% calcul de l'écart type
for colonne = 1:hauteur
     for ligne = 1:largeur
        for nbImages = 1:nbIterations
             ecartType(colonne,ligne) =
ecartType(colonne,ligne)+(((valeurPixel(colonne,ligne,nbImages)-
moyenne(colonne,ligne))^2)/nbIterations);
         end
         ecartType(colonne,ligne) = sqrt(ecartType(colonne,ligne));
     end
     %colonne
 end
```

Figure 4 : Calcul de l'écart-type

Après avoir expliqué le calcul de la moyenne et de l'écart-type, voici ce que nous obtenons graphiquement.



Figure 5 : Résultat graphique de la moyenne des 614 images

Comme nous pouvoir le voir sur l'image représentant la moyenne, la route est totalement visible. Tous les véhicules semblent avoir disparu. Cela est logique car sur nos 614 images, il y a plus souvent des pixels de route vide que des pixels de route avec une voiture par-dessus. De ce fait, en faisant la moyenne, ce sont les pixels de route vide qui priment sur les pixels de route avec une voiture dessus.



Figure 6 : Résultat graphique de l'écart-type des 614 images

Pour rappel, l'écart-type est une mesure de la dispersion ou de la variabilité des données dans un ensemble de données. Comme nous pouvoir le voir sur l'image représentant l'écart-type, les deux routes semblent être assez grises et le fond du décor est totalement noir.

En effet concernant le décor qui se situe sur les côtés de la route, il y a très peu de changements entre les différentes images. De ce fait, l'écart-type qui mesure la dispersion trouvera des valeurs très faibles (pour donner un ordre de grandeur, cela correspond à des valeurs inférieures à 10), ces pixels seront alors représentés en noir (0 étant le noir total).

Concernant les deux routes, il y a des nombreuses variations car des véhicules passent dessus ainsi que leurs ombres qui sont projetées. De ce fait, l'écart-type qui mesure la dispersion trouvera des valeurs moyennes voire grandes, ces pixels seront alors représentés en gris (l'image est en niveau de gris).

1.1.2 Classification des pixels en fond / voiture

Afin de classifier les pixels en fond / voiture de chacune des 614 images, nous avons créé des conditions sur la moyenne ainsi que sur l'écart-type.

Notre première condition concerne l'écart-type, nous avons donc défini que si l'écart-type est supérieur à 20, alors le pixel serait colorié en blanc. La catégorie blanche représente les pixels de route et la catégorie noire les pixels de décor. Ainsi si l'écart-type est supérieur à 20. Cela signifie qu'il y a une dispersion importante des données pour ce pixel. De ce fait, nous avons décidé de colorier ce pixel en blanc et dans le cas contraire ou l'écart-type est inférieur ou égal à 20, le pixel sera colorié en noir.

```
for colonne = 1:hauteur
    for ligne = 1:largeur
        if(ecartType(colonne,ligne)>20)
            imageFiltree(colonne,ligne) = 255;
        else
            imageFiltree(colonne,ligne) = 0;
        end
    end
end
```

Figure 7 : Code de classification des pixels en fond / voiture en fonction de l'écart-type



Figure 8 : Résultat de la classification avec l'écart-type uniquement

Comme nous pouvons le voir, avec la classification uniquement avec l'écart-type, nous obtenons une image binaire qui ressemble à la Figure 6 mais de manière accentuée.

Puis nous avons rajouté à celle précédente une condition en fonction de la moyenne. Parmi les pixels blancs de l'écart-type qui concerne les routes. Nous avons défini que si la valeur absolue de la moyenne du pixel est supérieure de 20 à la valeur absolue du pixel sur l'image en question, alors nous mettrons ce pixel en noir. Cela permet donc de colorier les pixels avec des forts changements car ils n'ont pas une moyenne stable :

```
 for colonne = 1:hauteur
    for ligne = 1:largeur
        if(ecartType(colonne,ligne)>20)
            imageFiltree(colonne,ligne) = 255;
            if(abs(uint8(moyenne(colonne,ligne))-I(colonne,ligne))>20)
                imageFiltree(colonne,ligne) = 0;
            end
        else
            imageFiltree(colonne,ligne) = 0;
        end
    end
end
```

Figure 9 : Code de classification des pixels en fond / voiture en fonction de l'écart-type et de la moyenne

Voici les différents résultats que nous obtenons pour la classification des pixels en fond / voiture en fonction de l'écart-type et de la moyenne :



Figure 10 : Image d'origine à gauche et image classifiée à droite (image_001)



Figure 11 : Image d'origine à gauche et image classifiée à droite (image_101)



Figure 12 : Image d'origine à gauche et image classifiée à droite (image_201)

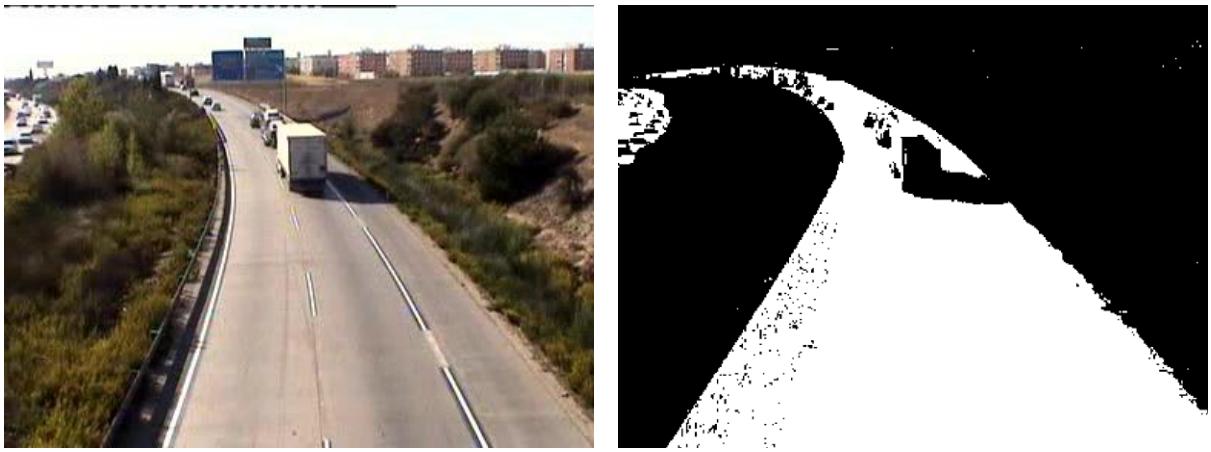


Figure 13 : Image d'origine à gauche et image classifiée à droite (image_301)



Figure 14 : Image d'origine à gauche et image classifiée à droite (image_401)



Figure 15 : Image d'origine à gauche et image classifiée à droite (image_501)



Figure 16 : Image d'origine à gauche et image classifiée à droite (image_601)

Nous pouvons voir que les résultats sont globalement satisfaisants. Nous pouvons également changer les seuils pour l'écart-type et la moyenne afin d'accentuer ou de diminuer cette classification.

1.2 Analyse de la variation du niveau de gris du pixel (200,150). Identification des plateaux dans les niveaux de gris de pixels pour faire l'estimation de la moyenne et de l'écart-type

1.2.1 Analyse de la variation du niveau de gris du pixel (200,150)

Afin d'analyser au mieux la variation du niveau de gris du pixel aux coordonnées 200 en X et 150 en Y, nous avons retracé le graphique :

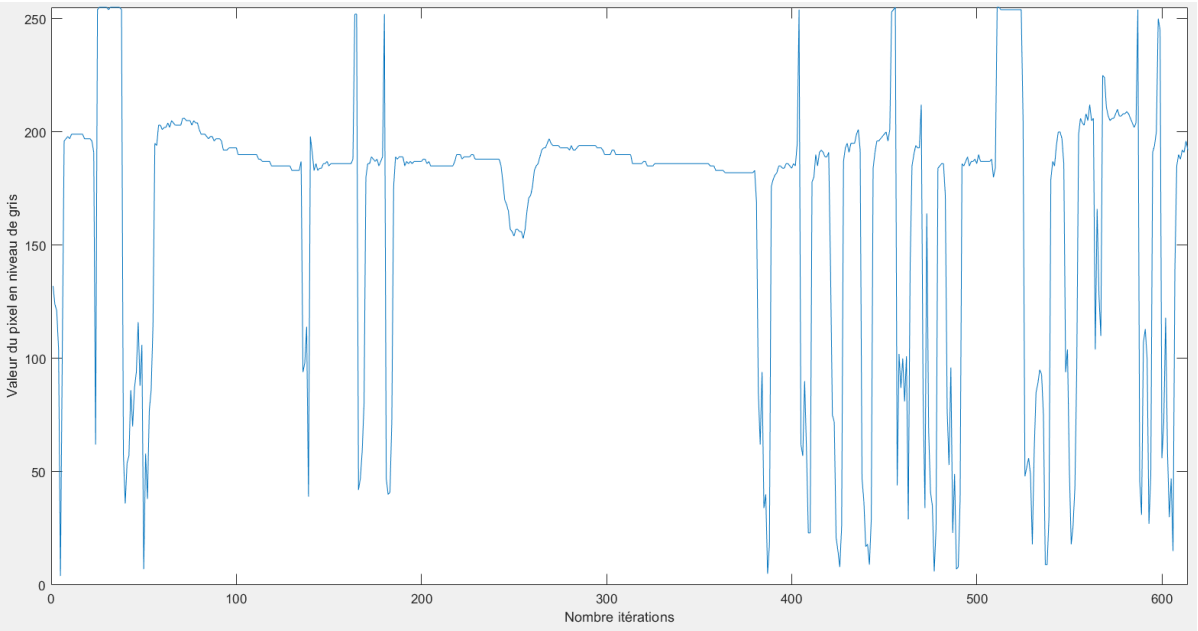


Figure 17 : Variation du niveau de gris du pixel (200,150)

Comme nous pouvons le voir sur ce graphique, les variations du niveau de gris semblent être assez importantes. En effet, la plage de valeur est d'environ 2 jusqu'à 255. Cela est dû au fait que le pixel (200,150) se trouve être sur la zone avec la route. De ce fait, lorsque ce pixel correspond à l'ombre d'un véhicule, sa valeur est alors proche de 255 (c'est-à-dire du noir). De plus, lorsque ce pixel correspond aux véhicules (souvent de couleur blanche ou colorée c'est-à-dire une autre couleur que le noir), le pixel est alors proche de 0. Cela explique les grandes variations que nous voyons.

Dans ce cas précis, la valeur de la moyenne équivaut à 165 et la valeur de l'écart-type est de 48.85. En s'intéressant sur la valeur de la moyenne, nous remarquons qu'elle est proche des valeurs des plateaux. Les plateaux correspondent aux zones avec des variations faibles. En effet, sur le graphique, les plateaux ont des valeurs aux alentours de 190. De ce fait, le calcul de la moyenne et de l'écart-type peut être assez trompeur

Pour la suite, afin de calculer au mieux la moyenne et l'écart-type, il serait alors intéressant de supprimer ces zones de fortes variations et de ne garder que les plateaux.

1.2.2 Identification des plateaux dans les niveaux de gris de pixels pour faire l'estimation de la moyenne et de l'écart-type

Je vais maintenant expliquer la méthode que nous avons utilisée pour identifier les plateaux dans les niveaux de gris de pixels et pour ensuite refaire l'estimation de la moyenne et de l'écart-type.

Comme nous le savons, chaque pixel est en niveau de gris. De ce fait sa valeur est comprise entre 0 et 255. Comme il y a 614 images, un seul pixel aura 614 niveaux de gris compris entre 0 et 255.

Notre idée a alors été de récupérer ces 614 niveaux de gris compris entre 0 et 255 et de les stocker dans un tableau de 256 éléments. Dans ce tableau de 256 éléments, chaque case correspondra au nombre d'occurrences du niveau gris. Pour exemple, si pour le pixel (0,0), parmi les 614 images, 30 d'entre elles ont pour valeur 0 en niveau de gris, alors la case 0 de notre tableau contiendra 30.

À partir de ce tableau et en nous inspirant de la distribution gaussienne, nous voulions supprimer les zones avec de faibles valeurs ce qui correspond aux extrémités de la distribution gaussienne (à partir de 2 σ ou 3 σ) :

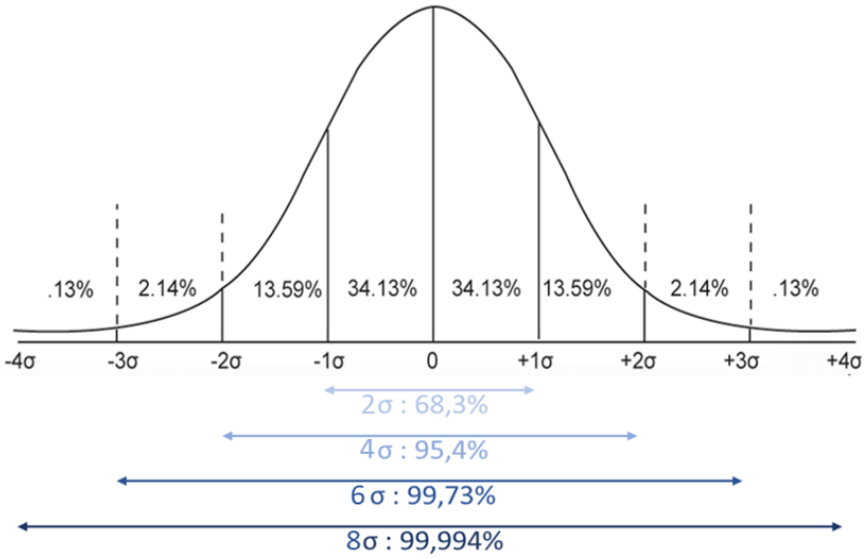


Figure 18 : Distribution gaussienne

Voici ce à quoi ressemblait notre code. La première partie sert à charger et à convertir les images en niveau de gris. La suite permet comme dit précédemment de créer notre tableau où chaque case correspondra au nombre d'occurrences du niveau gris :

```
for compteurIteration = 1:nbIterations
    if(compteurIteration < 10)
        I = imread(['image_00' num2str(compteurIteration, '%01d') '.jpg']); %
Charger l'image
    end

    if(compteurIteration >= 10 && compteurIteration < 100)
        I = imread(['image_0' num2str(compteurIteration, '%02d') '.jpg']);
    end

    if(compteurIteration >= 100)
        I = imread(['image_' num2str(compteurIteration, '%02d') '.jpg']);
    end

    if ndims(I) == 3
        I = rgb2gray(I); % Convertir en niveaux de gris
    end

    for colonne = 1:hauteur
        for ligne = 1:largeur
            imageChargee(compteurIteration,colonne,ligne) = I(colonne,ligne);
        end
    end
end

tableauGaussSomme = zeros(hauteur,largeur);

for colonne = 1:hauteur
    for ligne = 1:largeur
        tableauPixel = zeros(1,nbIterations);
        tableauGauss = zeros(1,256);

        for compteurIteration = 1:nbIterations
            tableauPixel(compteurIteration) =
imageChargee(compteurIteration,colonne,ligne);
        end

%         figure;
%         plot(tableauPixel);
%         ylim([0, 255]);
%         xlim([0, nbIterations]);
%         xlabel('Nombre itérations')
%         ylabel('Valeur du pixel en niveau de gris')
        for cptIt = 1:nbIterations
            tableauGauss(tableauPixel(cptIt)+1) =
tableauGauss(tableauPixel(cptIt)+1)+1;
        end
```

Figure 19 : Code de chargement et conversion des images en niveau de gris puis création du tableau où chaque case correspondra au nombre d'occurrences du niveau gris

Voici les différents résultats graphiques que nous avons vus en affichant ce tableau où chaque case correspondra au nombre d'occurrences du niveau gris :

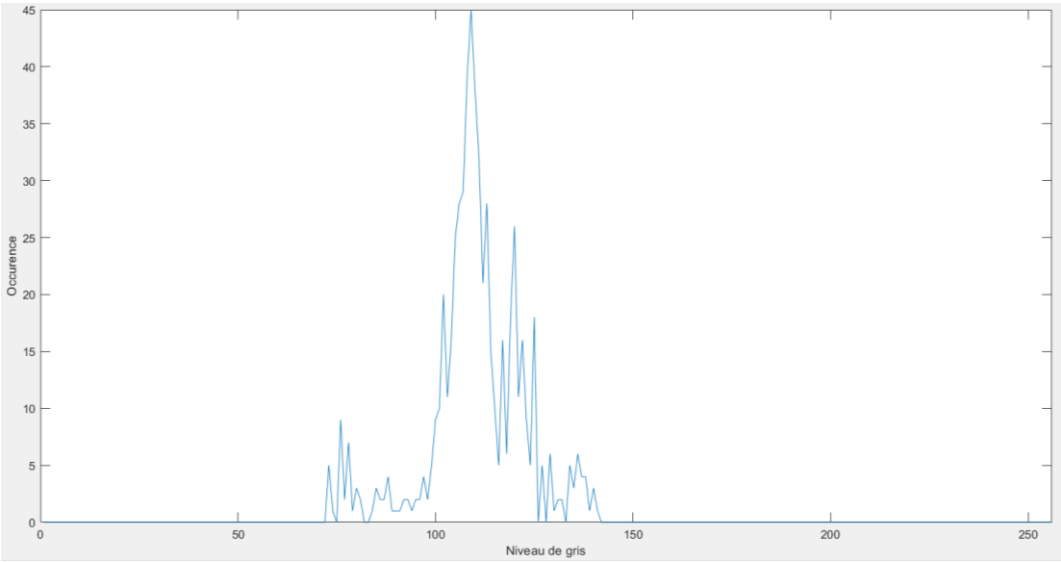


Figure 20 : Affichage du tableau pour le pixel (50,50)

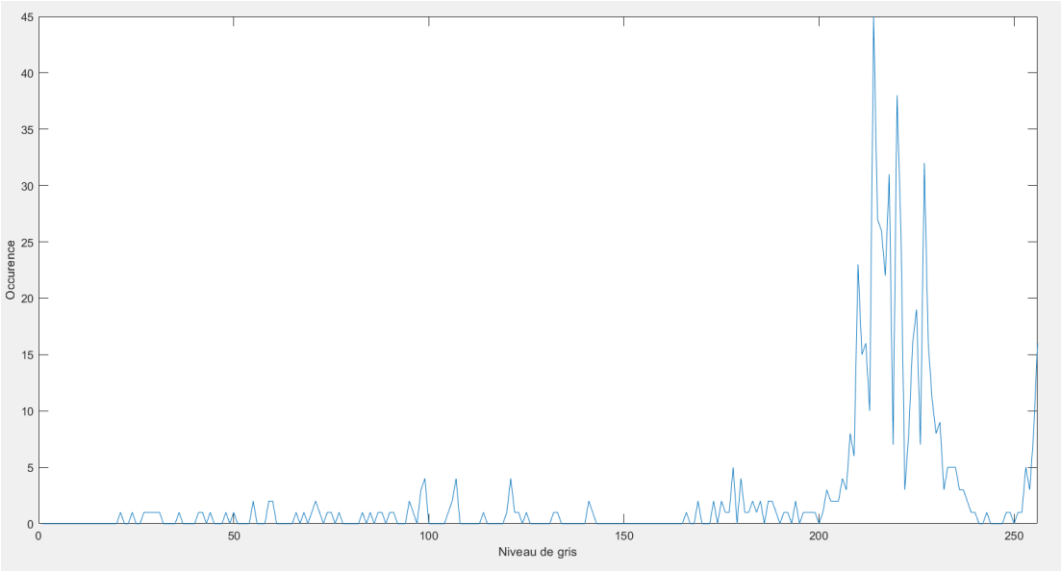


Figure 21 : Affichage du tableau pour le pixel (150,150)

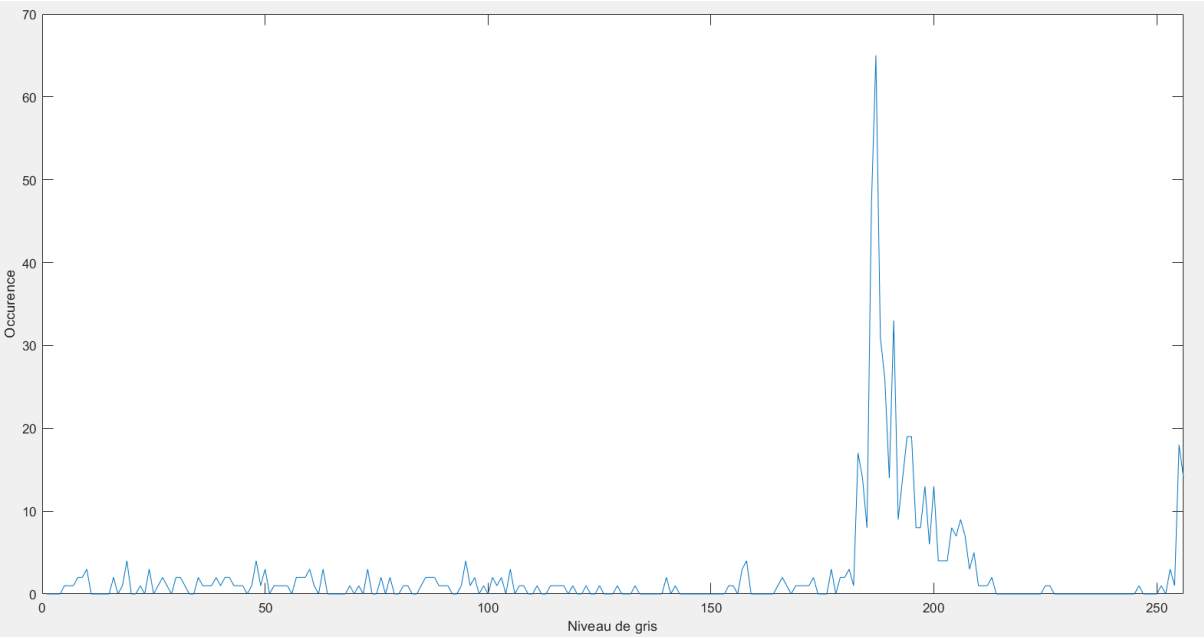


Figure 22 : Affichage du tableau pour le pixel (200,150)

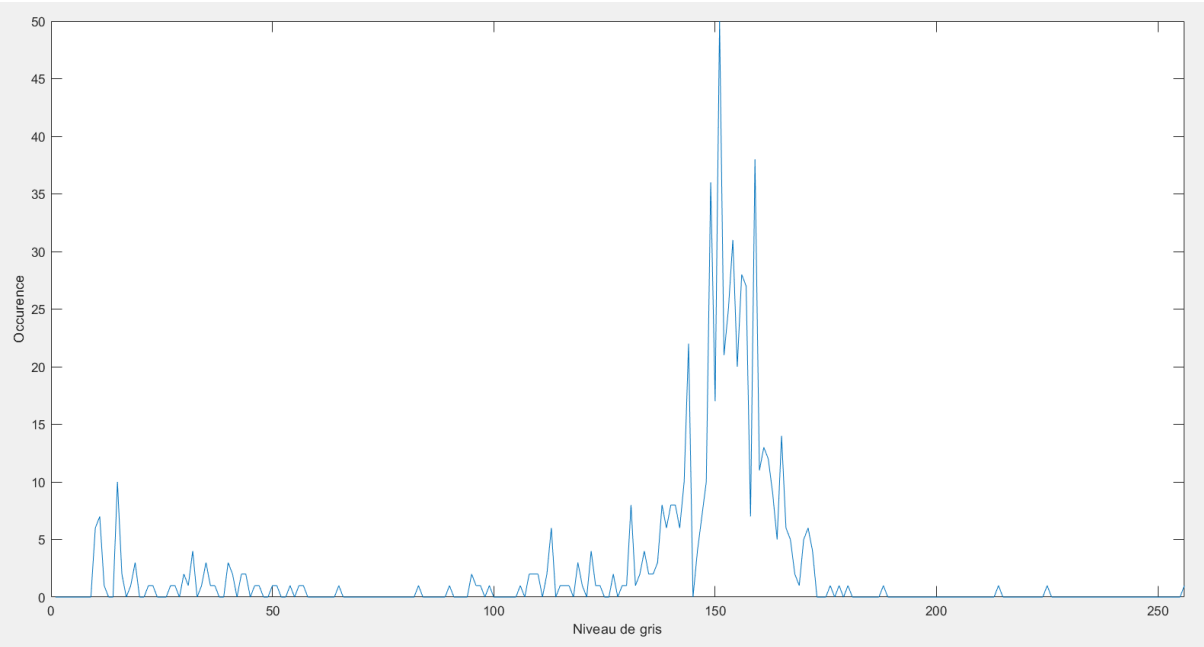


Figure 23 : Affichage du tableau pour le pixel (200,200)

À partir ces graphiques et donc du tableau, nous pouvons définir une valeur seuil permettant de filtrer les grandes variations. Nous pouvons par exemple définir que tous les niveaux de gris dont l'occurrence n'est pas au moins de 30% celle la valeur de l'occurrence la plus forte, alors ces variations sont supprimées.

Pour expliquer la démarche sur le dernier graphique, nous remarquons que le pixel (200,200) sur les 614 images a le plus souvent comme niveau de gris la valeur environ 152 qui apparaît 50 fois. De ce fait, nous voulons garder uniquement les autres valeurs de niveau de gris qui apparaissent au moins 0.3*50 = 15 fois.

Ainsi, avec ce seuil, cela permet de réduire les variations non souhaitées et de ne garder que les zones avec le plus d'itérations.

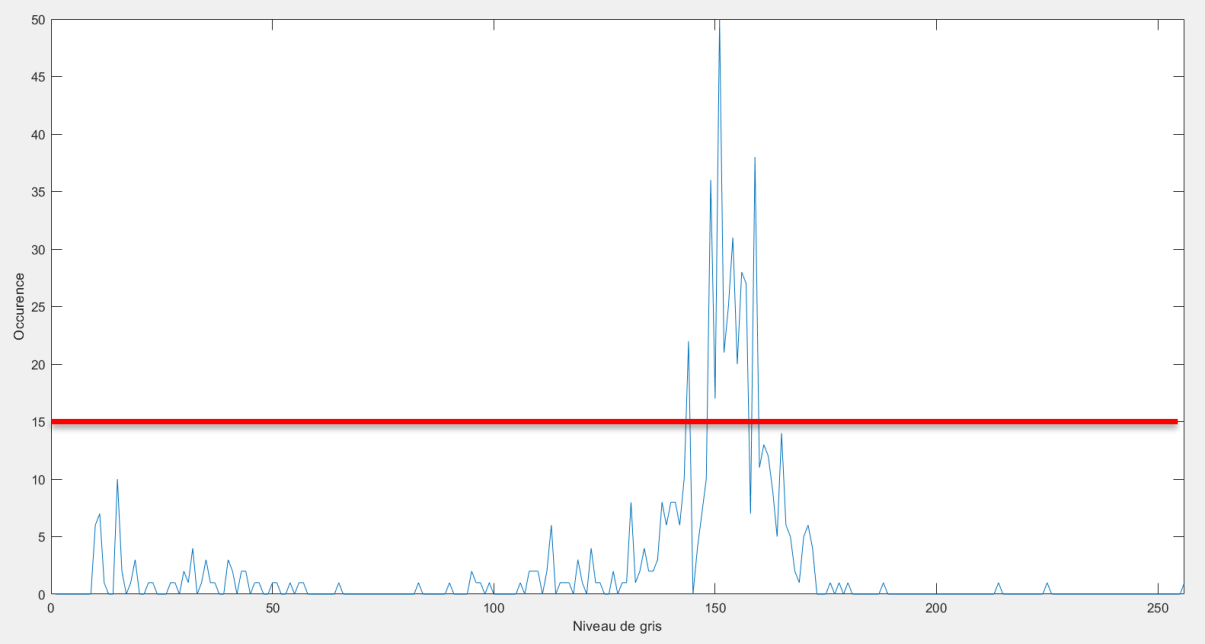


Figure 24 : Seuillage des variations

Ainsi, tous les éléments au-dessus de la droite rouge seront conservés alors que les éléments au-dessous seront supprimés. Voici ce à quoi cela correspond au niveau du code :

```
tableauGaussSomme(colonne,ligne) = sum(tableauGauss);


% figure;
% plot(tableauGauss);
% xlim([0, 256]);
% xlabel('Niveau de gris')
% ylabel('Occurence')

maxTableau = max(tableauGauss);
seuil = maxTableau*0.5;
nbElementMoyenne = 0;
moy = 0

for compteurSeuil = 1:256
    if(tableauGauss(compteurSeuil)>seuil)
        moy = moy + compteurSeuil*tableauGauss(compteurSeuil);
        nbElementMoyenne = nbElementMoyenne + tableauGauss(compteurSeuil);
    end
end
```

Figure 25 : Code permettant de seuiller les grandes variations

Puis nous pouvons calculer la moyenne à partir des éléments que nous souhaitons garder uniquement et recalculer l'écart-type à partir de la formule vue précédemment :

```
for compteurSeuil = 1:256
    if(tableauGauss(compteurSeuil)>seuil)
        moy = moy + compteurSeuil*tableauGauss(compteurSeuil);
        nbElementMoyenne = nbElementMoyenne + tableauGauss(compteurSeuil);
    end
end

moyenne(colonne,ligne) = (moy/nbElementMoyenne)-1;
ligne;
end

end


for colonne = 1:hauteur
     for ligne = 1:largeur
        for nbImages = 1:nbIterations
             ecartType(colonne,ligne) =
ecartType(colonne,ligne)+(((imageChargee(nbImages,colonne,ligne)-
moyenne(colonne,ligne))^2)/nbIterations);
         end
         ecartType(colonne,ligne) = sqrt(ecartType(colonne,ligne));
     end
 end
```

Figure 26 : Calcul de la nouvelle moyenne et du nouvel écart-type

```
seuil = maxTableau*0.5;
```

Figure 27 : Seuil de tolérance des variations

Voici la moyenne à gauche et l'écart-type à droite avec un seuil d'au moins 50% (de la valeur de l'occurrence la plus forte). Si la valeur de l'occurrence la plus forte vaut 100, alors tous les éléments inférieurs à 50 seront supprimés :



Figure 28 : Moyenne à gauche et écart-type à droite pour un seuil de 50%



Figure 29 : Moyenne à gauche et écart-type à droite pour un seuil de 25%



Figure 30 : Moyenne à gauche et écart-type à droite pour un seuil de 10%

1.3 Faites tourner un algorithme ΣΔ

Un algorithme ΣΔ ou Sigma-Delta est un type d'algorithme utilisé pour convertir un signal analogique en signal numérique en modulant la quantification du signal. Il fonctionne en mesurant la différence entre le signal analogique d'entrée et une version numérique approximative du signal, puis en utilisant cette différence pour ajuster la quantification du signal numérique. Cela permet de réduire le bruit dans le signal numérique et d'améliorer la précision de la conversion. En traitement d'image, l'algorithme ΣΔ peut également être utilisé pour la compression d'images ou pour la détection des différences entre deux images consécutives.

Après avoir parcouru le cours, nous n'avons pas trouvé l'algorithme ΣΔ et n'avons pas réussi à comprendre comment l'implémenter ainsi que son rôle vis-à-vis de l'exercice. De ce fait, sans plus d'informations provenant du professeur, nous avons décidé de ne pas traiter la question.

1.4 Séparation lors de la détection des voitures et de leurs ombres

1.4.1 Avec l'espace YCbCr

Un espace de représentation plus invariant aux effets des ombres est un espace de couleur ou de caractéristiques qui permet de détecter les objets présents dans une image en ignorant les variations dues aux ombres. Dans le contexte de l'exercice, cela signifie qu'il faut trouver un espace de représentation qui permet de distinguer les voitures des ombres des voitures en mouvement dans l'image.

Nous avons voulu nous tourner vers le modèle de couleurs YCbCr. Cet espace de couleur est également utile pour la détection d'objets, car il sépare les informations de luminance (Y) des informations de chrominance (Cb et Cr).

Après recherches concernant l'élimination des ombres, nous avons appris que les ombres des voitures ont tendance à avoir une luminance plus faible que les objets mobiles, et une différence de chrominance plus élevée par rapport aux objets mobiles. Ainsi, nous pouvons utiliser trois seuils basés sur la luminance et la chrominance bleue et rouge pour identifier les pixels correspondant aux ombres et les supprimer de l'image binaire. Voici ce à quoi correspond la conversion d'une image RGB en YCrCb :

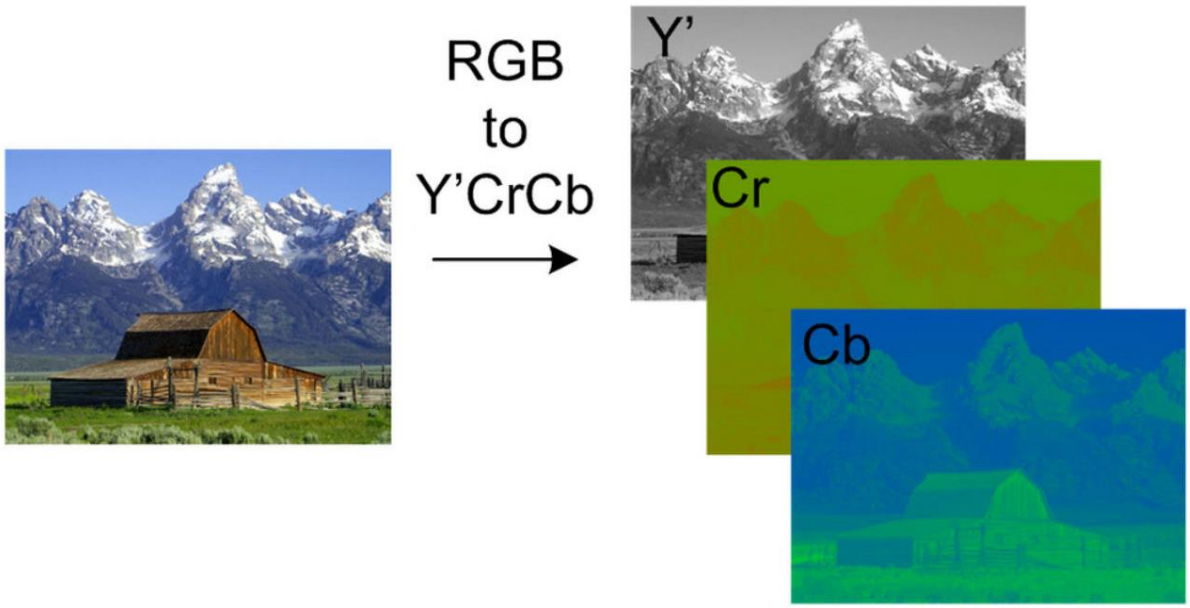


Figure 31 : Conversion RGB en YCrCb

Voici comment convertir une image RGB sur Matlab en YCrCb puis nous pouvons récupérer les informations concernant la luminance (Y), la chrominance rouge (Cr) et la chrominance bleue (Cb).

```
ycbcrImage = rgb2ycbcr(I);
luminance = ycbcrImage(:,:,1);
Cb = ycbcrImage(:,:,2);
Cr = ycbcrImage(:,:,3);
```

Figure 32 : Conversion d'une image RGB en YCrCb et récupération des 3 canaux d'information

En appliquant des valeurs seuils concernant les trois paramètres vus précédemment que nous ajoutons à ceux de la question 1.1.2 Classification des pixels en fond / voiture, nous pouvons supprimer les ombres :

```
for colonne = 1:hauteur
    for ligne = 1:largeur
        if(ecartType(colonne,ligne)>20)
            imageFiltree(colonne,ligne) = 255;
            if(abs(uint8(moyenne(colonne,ligne))-I(colonne,ligne))>20)
                imageFiltree(colonne,ligne) = 0;

if(luminance(colonne,ligne)<70&&luminance(colonne,ligne)>23&&Cb(colonne,ligne)>122&
&Cb(colonne,ligne)<142&&Cr(colonne,ligne)>122&&Cr(colonne,ligne)<133)
                    imageFiltree(colonne,ligne) = 255;
                end
            end
        else
            imageFiltree(colonne,ligne) = 0;
        end
    end
end
```

Figure 33 : Code de classifications des pixels en fond / voiture avec détection des ombres

Parmi ce code, voici la partie importante qui concerne le traitement sur l'image YCrCb où nous avons défini 6 seuils afin d'encadrer la luminance, Cb et Cr. Ces valeurs ont été trouvées de manière empirique :

```
if(luminance(colonne,ligne)<70&&luminance(colonne,ligne)>23&&Cb(colonne,ligne)>122&
&Cb(colonne,ligne)<142&&Cr(colonne,ligne)>122&&Cr(colonne,ligne)<133)
                    imageFiltree(colonne,ligne) = 255;
end
```

Figure 34 : Code des seuils de détection des ombres

Voici les différents résultats que nous avons obtenus :



Figure 35 : image classifiée à gauche et image classifiée avec détection des ombres à droite (image_001)

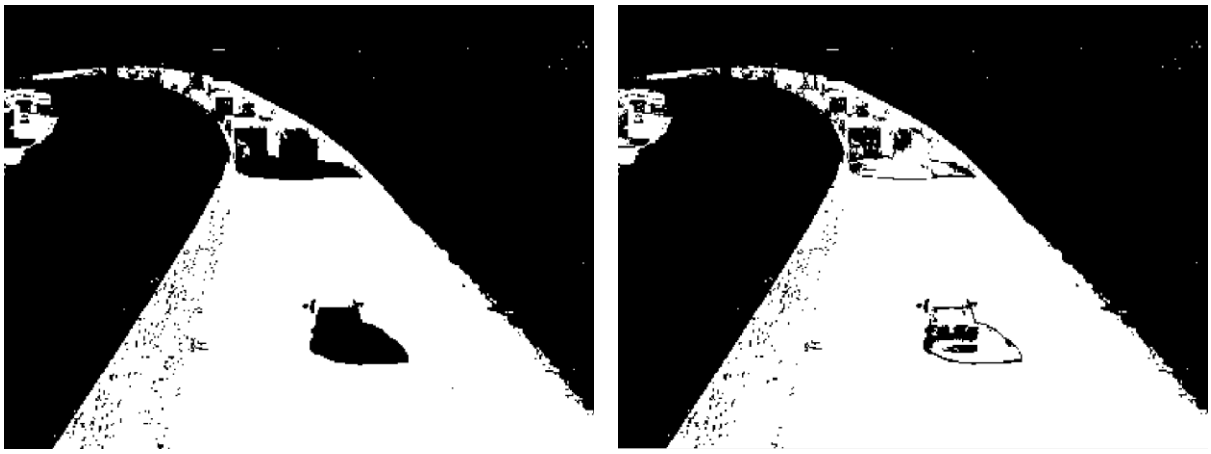


Figure 36 : image classifiée à gauche et image classifiée avec détection des ombres à droite (image_101)

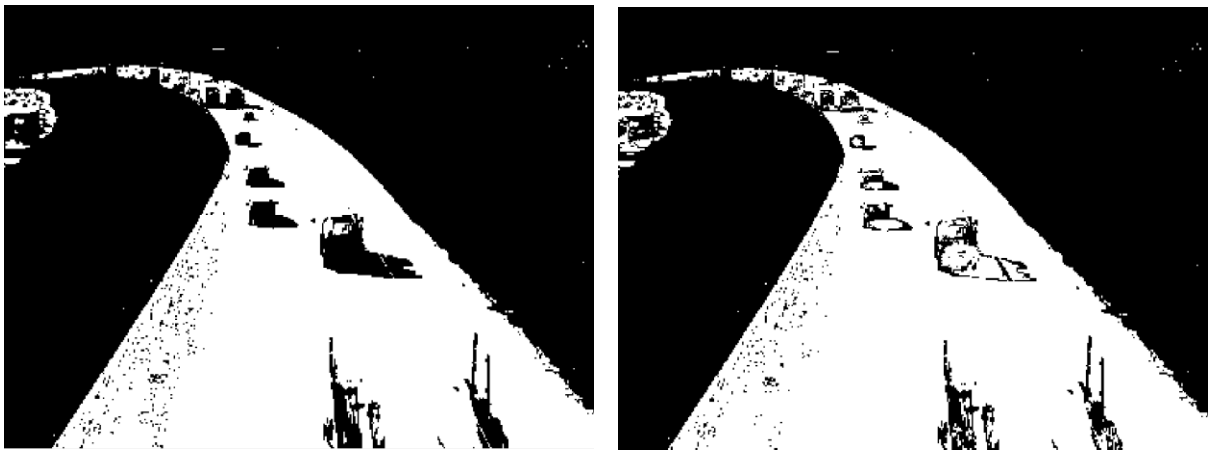


Figure 37 : image classifiée à gauche et image classifiée avec détection des ombres à droite (image_201)



Figure 38 : image classifiée à gauche et image classifiée avec détection des ombres à droite (image_301)

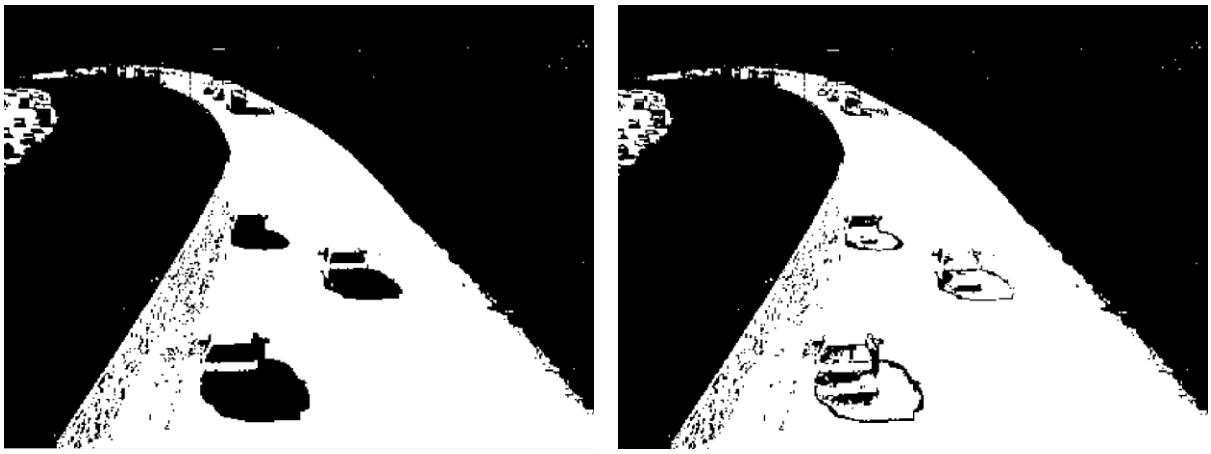


Figure 39 : image classifiée à gauche et image classifiée avec détection des ombres à droite (image_401)



Figure 40 : image classifiée à gauche et image classifiée avec détection des ombres à droite (image_501)

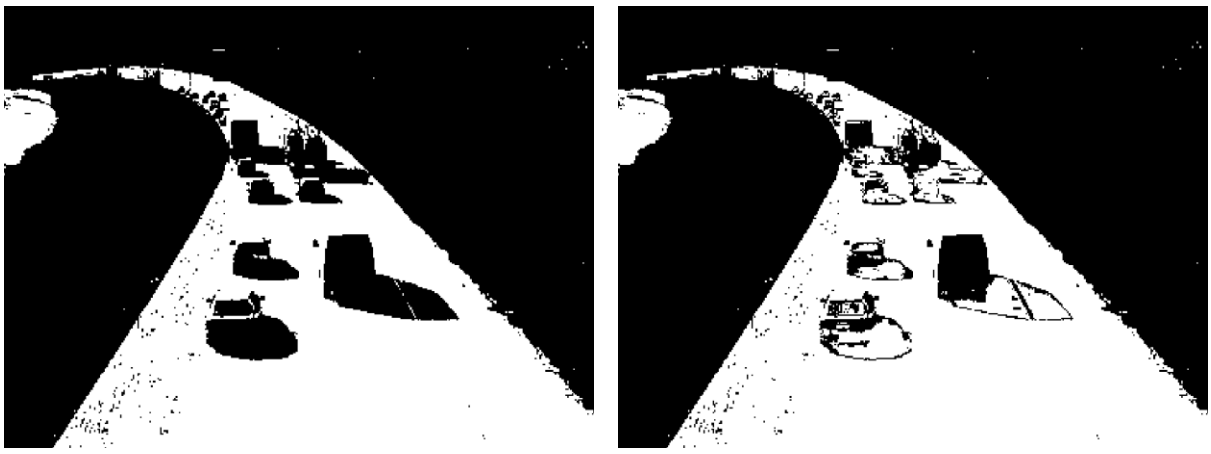


Figure 41 : image classifiée à gauche et image classifiée avec détection des ombres à droite (image_601)

Comme nous pouvons le voir, cela réduit une partie de l'ombre mais touche également les voitures. Les résultats sont corrects mais peuvent être améliorés. Ce qui influence

fortement les résultats est le seuil que nous choisissons concernant la luminance et particulièrement le fait que nous avons défini la luminance comme étant inférieure à 70.

1.4.2 Avec l'espace HSV fusionné à l'espace YCbCr

L'espace de couleur HSV (teinte, saturation, valeur) est un espace de couleur cylindrique qui représente les couleurs en fonction de trois dimensions : la teinte (hue), la saturation (saturation) et la valeur (value ou brightness).
- La teinte est une propriété de la couleur qui décrit l'emplacement de la couleur sur le cercle chromatique. Elle est mesurée en degrés, allant de 0 à 360 degrés, et peut être représentée par une couleur pure (rouge, vert, bleu, etc.) ou par une couleur mixte (orange, violet, turquoise, etc.).
- La saturation mesure la pureté de la couleur et représente la quantité de gris dans la couleur. Elle est généralement mesurée en pourcentage, allant de 0 % (gris) à 100 % (couleur pure).
- La valeur mesure la luminosité de la couleur. Elle est généralement mesurée en pourcentage, allant de 0 % (noir) à 100 % (blanc).

L'espace de couleur HSV est souvent utilisé dans le traitement d'images pour séparer les informations de couleur et de luminosité, ainsi que pour faciliter la manipulation de couleurs spécifiques.

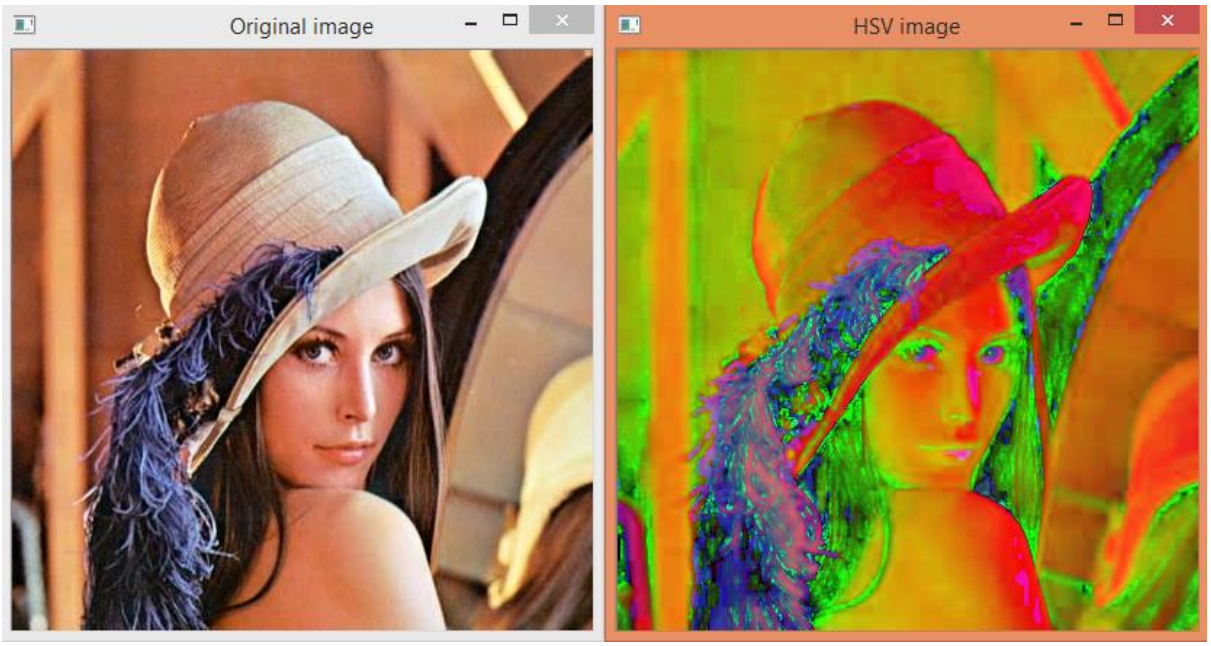


Figure 42 : Conversion RGB en HSV

Voici comment convertir une image RGB sur Matlab en HSV puis nous pouvons récupérer les informations concernant la teinte (H), la saturation (S) et la valeur (V).

```
hsvImage = rgb2hsv(I);
H = hsvImage(:,:,1);
S = hsvImage(:,:,2);
V = hsvImage(:,:,3);
```

Figure 43 : Conversion d'une image RGB en YCrCb et récupération des 3 canaux d'information

En appliquant des valeurs seuils concernant les trois paramètres vus précédemment que nous ajoutons à ceux de la question 1.1.2 Classification des pixels en fond / voiture, nous pouvons supprimer les ombres :

```
for colonne = 1:hauteur
    for ligne = 1:largeur
        if(ecartType(colonne,ligne)>20)
            imageFiltree(colonne,ligne) = 255;
            if(abs(uint8(moyenne(colonne,ligne))-I(colonne,ligne))>20)
                imageFiltree(colonne,ligne) = 0;

if(H(colonne,ligne)>0.4&&H(colonne,ligne)<0.90&&S(colonne,ligne)<0.5&&S(colonne,lig
ne)>0.1&&V(colonne,ligne)<0.4&&V(colonne,ligne)>0.05)
                    imageFiltree(colonne,ligne) = 255;
                end

if(luminance(colonne,ligne)<70&&luminance(colonne,ligne)>23&&Cb(colonne,ligne)>122&
&Cb(colonne,ligne)<142&&Cr(colonne,ligne)>122&&Cr(colonne,ligne)<133)
                    imageFiltree(colonne,ligne) = 255;
                end
            end
        else
            imageFiltree(colonne,ligne) = 0;
        end
    end
end
```

Figure 44 : Code de classifications des pixels en fond / voiture avec détection des ombres

Parmi ce code, voici la partie importante qui concerne le traitement sur l'image HSV où nous avons défini 6 seuils afin d'encadrer la teinte (H), la saturation (S) et la valeur (V). Ces valeurs ont été trouvées de manière empirique :

```
if(H(colonne,ligne)>0.4&&H(colonne,ligne)<0.90&&S(colonne,ligne)<0.5&&S(colonne,lig
ne)>0.1&&V(colonne,ligne)<0.4&&V(colonne,ligne)>0.05)
                    imageFiltree(colonne,ligne) = 255;
end
```

Figure 45 : Code des seuils de détection des ombres

Voici les différents résultats que nous avons obtenus :

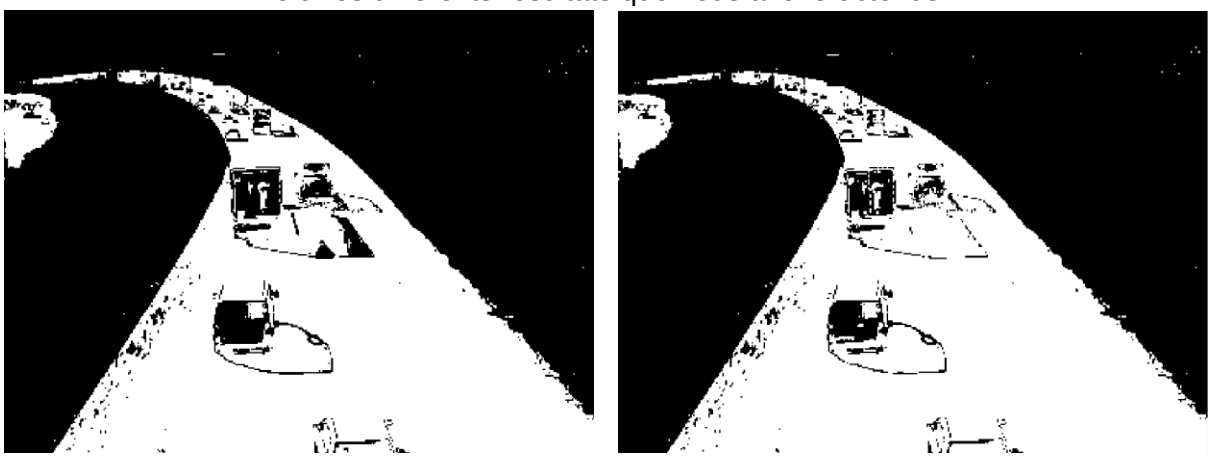


Figure 46 : Image classifiée avec détection des ombres à gauche (via YcbCr) et image classifiée avec détection des ombres à droite via YcbCr et HSV (image_001)

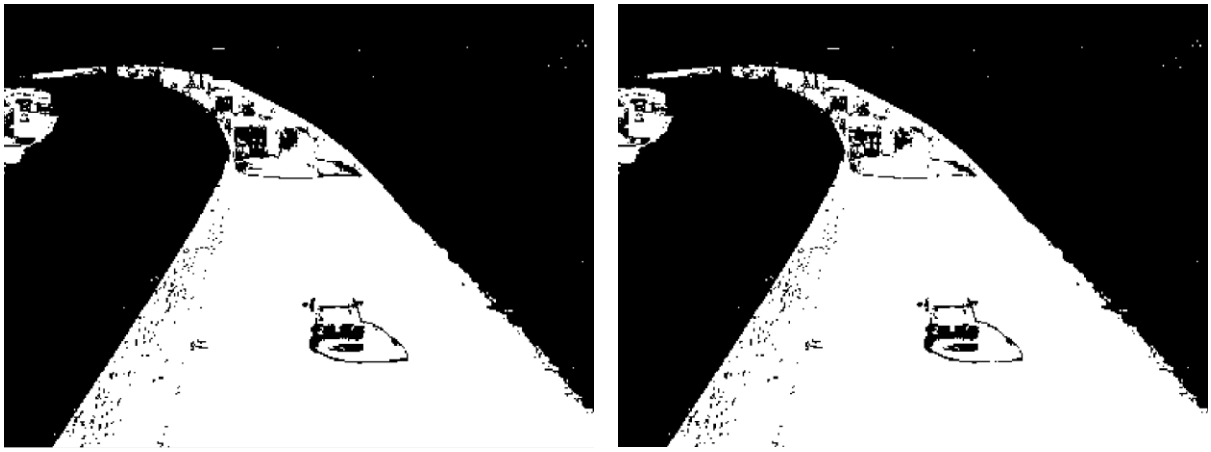


Figure 47 : Image classifiée avec détection des ombres à gauche (via YcbCr) et image classifiée avec détection des ombres à droite via YcbCr et HSV (image_101)

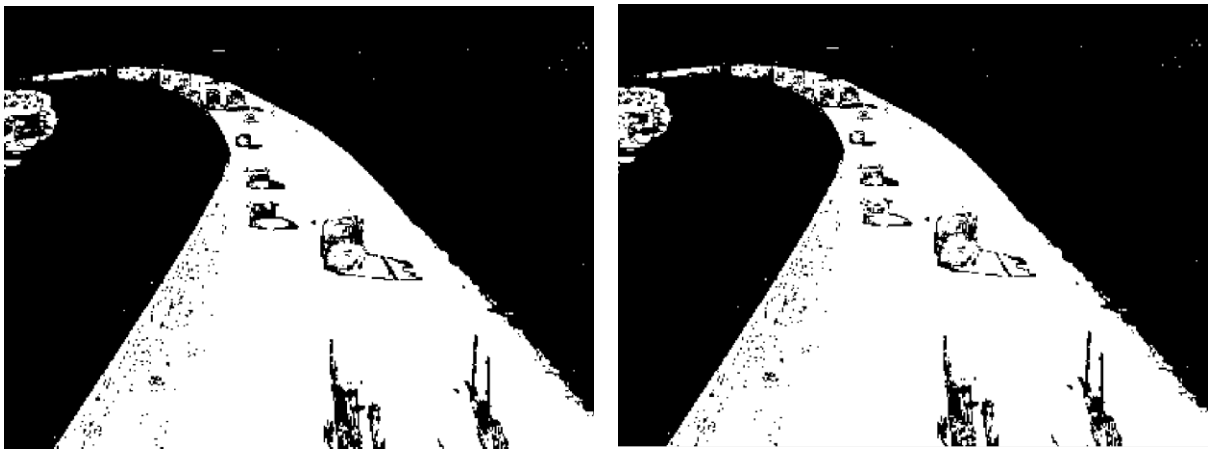


Figure 48 : Image classifiée avec détection des ombres à gauche (via YcbCr) et image classifiée avec détection des ombres à droite via YcbCr et HSV (image_201)

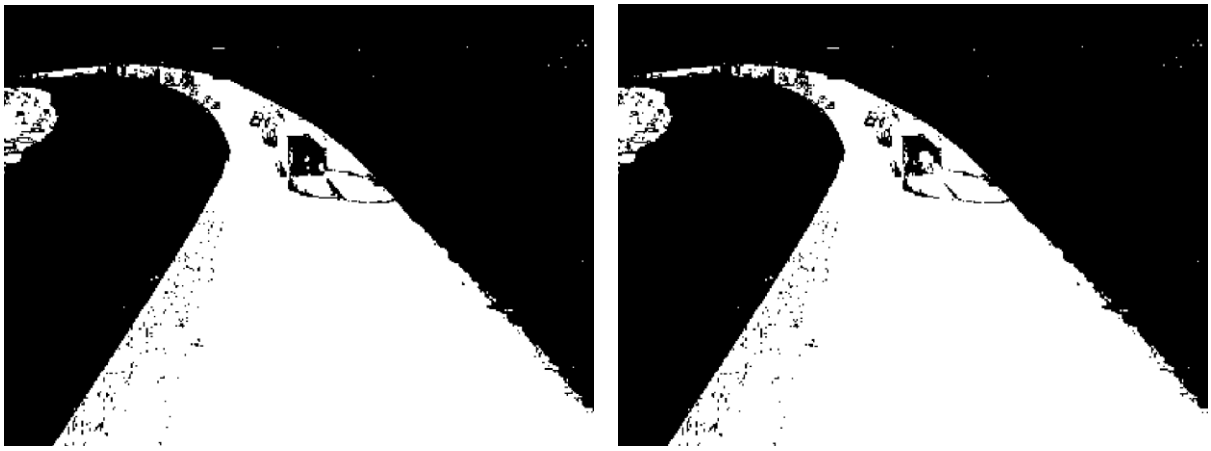


Figure 49 : Image classifiée avec détection des ombres à gauche (via YcbCr) et image classifiée avec détection des ombres à droite via YcbCr et HSV (image_301)



Figure 50 : Image classifiée avec détection des ombres à gauche (via YcbCr) et image classifiée avec détection des ombres à droite via YcbCr et HSV (image_401)



Figure 51 : Image classifiée avec détection des ombres à gauche (via YcbCr) et image classifiée avec détection des ombres à droite via YcbCr et HSV (image_501)

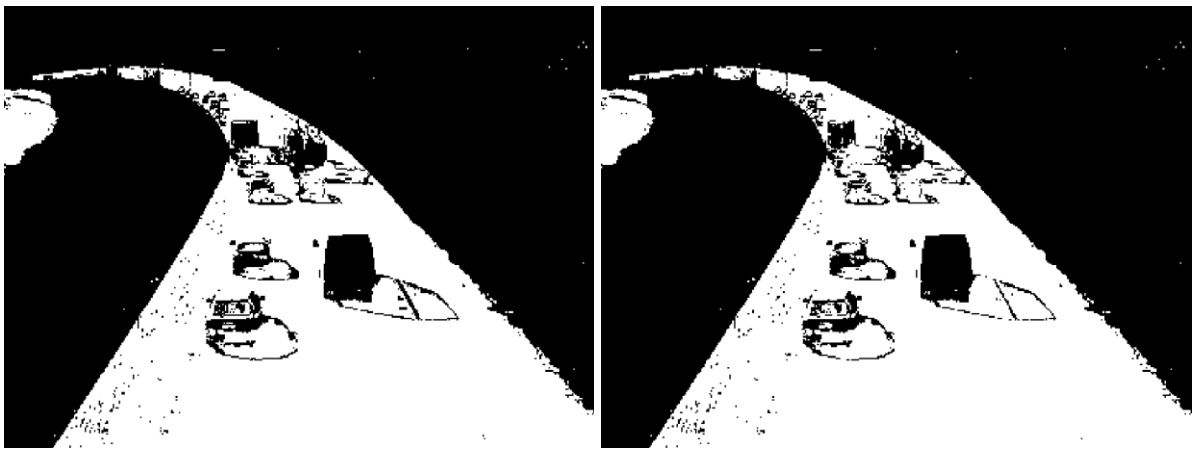


Figure 52 : Image classifiée avec détection des ombres à gauche (via YcbCr) et image classifiée avec détection des ombres à droite via YcbCr et HSV (image_601)

Comme nous pouvons le voir, avec la fusion des seuils sur ces deux espaces, cela réduit davantage les ombres des voitures. Les résultats sont meilleurs que précédemment mais il reste des améliorations à y faire. Notamment en se penchant sur les valeurs des seuils pour ces deux espaces. Au total, cela fait plus de 12 paramètres que nous pouvons ajuster.

2. Flot optique

2.1 Introduction

L'article "Détermination du flux optique" de Berthold K.P. Horn et Brian G. Schunck décrit un algorithme pour calculer le flux optique, c'est-à-dire la vitesse à laquelle les pixels dans une image se déplacent par rapport à une autre image.

L'algorithme proposé par les auteurs utilise une méthode de minimisation de l'erreur pour estimer le flux optique. L'idée est de trouver une estimation du flux optique qui minimise l'erreur entre les deux images en utilisant une fonction de coût qui mesure la différence entre les deux images.

Les auteurs présentent des expériences pour évaluer la performance de leur algorithme en utilisant des séquences d'images synthétiques et réelles. Les résultats montrent que l'algorithme est capable de fournir des estimations précises du flux optique.

2.2 Explication de l’algorithme

Après lecture de l’article, voici les différentes étapes de l'algorithme que nous avons relevé pour estimer le flux optique. Nous avons au total 6 grandes étapes :

- Prétraitement des images : Les images en entrée sont prétraitées pour réduire le bruit et augmenter le contraste. Les auteurs utilisent une méthode de convolution avec un noyau gaussien pour flouter légèrement les images et ainsi réduire le bruit, puis appliquent une transformation de contraste pour augmenter la différence entre les niveaux de gris dans les images
- Calcul des gradients : On calcule les gradients des deux images en utilisant un opérateur de détection de bord. Les gradients donnent l'orientation et l'intensité des variations de luminosité dans les images
- Estimation initiale du flux optique : On estime une première approximation du flux optique en utilisant une méthode de correspondance de points d'intérêt. Cette étape fournit une estimation initiale de la vitesse à laquelle les points dans l'image se déplacent
- Minimisation de l'erreur : On minimise l'erreur entre les deux images en utilisant une fonction de coût qui mesure la différence entre les gradients des images. L'idée est de trouver une estimation du flux optique qui minimise cette erreur. Pour cela, on utilise une méthode de régularisation pour contrôler la régularité spatiale du flux optique, c'est-à-dire pour éviter des variations brusques dans le flux optique d'un pixel à l'autre
- Itération : Puis l’on répète les étapes 3 et 4 afin d’affiner l'estimation du flux optique en utilisant la nouvelle estimation du flux optique obtenue à chaque itération. Cette étape permet d'améliorer la précision de l'estimation du flux optique
- Post-traitement : Finalement, on applique un post-traitement pour supprimer les estimations de flux optique aberrantes (par exemple, en utilisant un seuil pour éliminer les estimations qui sont trop grandes ou trop petites). Il est également possible d’utiliser une méthode de lissage pour rendre les estimations du flux optique plus régulières

En conclusion, cet algorithme utilise une approche de minimisation d'erreur pour estimer le flux optique à partir de deux images en entrée, en utilisant une méthode de régularisation pour garantir une estimation régulière et stable. L'itération est utilisée dans le bus d’affiner progressivement l'estimation du flux optique.

2.3 Utilité du paramètre α

Le paramètre α dans l'algorithme proposé est un facteur de pondération utilisé dans la fonction de coût pour contrôler l'importance de la régularisation spatiale du flux optique. Plus la valeur de α est élevée, plus la régularisation spatiale est importante dans l'estimation du flux optique. À l'inverse, plus la valeur de α est faible, moins la régularisation spatiale est importante, ce qui permet à l'estimation du flux optique d'être plus sensible aux variations locales.

Si l'on fixe la valeur du paramètre α à 10, cela signifierait que la régularisation spatiale est très importante dans l'estimation du flux optique. Cela peut avoir pour effet de rendre l'estimation du flux optique plus lisse et régulière, mais au détriment de la précision locale. En d'autres termes, l'estimation du flux optique peut manquer de détails fins et de variations locales.

La valeur optimale du paramètre α dépend des caractéristiques spécifiques des images en entrée et des objectifs de l'application. Dans l'article, les auteurs ont expérimenté avec différentes valeurs de α comprise entre 1 et 10 et ont constaté que cela donnait des résultats satisfaisants pour une variété de scénarios.

3. Annexes

3.1 CodeMatlab

```
nbIterations = 614;
compteurIteration = 1;
hauteur = 288;
largeur = 384;
valeurPixel = zeros(hauteur,largeur,nbIterations);
ecartType = zeros(hauteur,largeur);
moyenne = zeros(hauteur,largeur,'int32');
imageFiltree = zeros(hauteur,largeur);
imageChargee = zeros(nbIterations, hauteur,largeur);

% ---------------------------------- Q1 ----------------------------------
while (compteurIteration < nbIterations)

    if(compteurIteration < 10)
        I = imread(['image_00' num2str(compteurIteration, '%01d') '.jpg']); %
Charger l'image
    end

    if(compteurIteration >= 10 && compteurIteration < 100)
        I = imread(['image_0' num2str(compteurIteration, '%02d') '.jpg']); %
Charger l'image
    end

    if(compteurIteration >= 100)
        I = imread(['image_' num2str(compteurIteration, '%02d') '.jpg']); % Charger
l'imagew
    end

    if ndims(I) == 3
        I = rgb2gray(I); % Convertir en niveaux de gris
    end

    [hauteur, largeur] = size(I);

    for colonne = 1:hauteur
        for ligne = 1:largeur
            moyenne(colonne,ligne) =
moyenne(colonne,ligne)+int32(I(colonne,ligne));
            valeurPixel(colonne,ligne,compteurIteration) = I(colonne,ligne);
        end
    end

    compteurIteration = compteurIteration + 1;
end

%calcul de la moyenne
for colonne = 1:hauteur
    for ligne = 1:largeur
        moyenne(colonne,ligne)=moyenne(colonne,ligne)/(nbIterations-1);
    end
end

% calcul de l'écart type
for colonne = 1:hauteur
     for ligne = 1:largeur
        for nbImages = 1:nbIterations
             ecartType(colonne,ligne) =
ecartType(colonne,ligne)+(((valeurPixel(colonne,ligne,nbImages)-
moyenne(colonne,ligne))^2)/nbIterations);
         end
         ecartType(colonne,ligne) = sqrt(ecartType(colonne,ligne));
```

```
     end
     %colonne
 end
%
%
%
% % ----------------- Q1 FILTREE ----------------------------
compteurIteration = 1;
while (compteurIteration < nbIterations)
    if(compteurIteration < 10)
        I = imread(['image_00' num2str(compteurIteration, '%01d') '.jpg']); %
Charger l'image
    end

    if(compteurIteration >= 10 && compteurIteration < 100)
        I = imread(['image_0' num2str(compteurIteration, '%02d') '.jpg']);
    end

    if(compteurIteration >= 100)
        I = imread(['image_' num2str(compteurIteration, '%02d') '.jpg']);
    end

    ycbcrImage = rgb2ycbcr(I);
    luminance = ycbcrImage(:,:,1);
    Cb = ycbcrImage(:,:,2);
    Cr = ycbcrImage(:,:,3);
    hsvImage = rgb2hsv(I);
    H = hsvImage(:,:,1);
    S = hsvImage(:,:,2);
    V = hsvImage(:,:,3);

%     figure;
%     imshow(ycbcrImage);
%     figure;
%     imshow(luminance);
%     figure;
%     imshow(Cb);
%     figure;
%     imshow(Cr);

    if ndims(I) == 3
        I = rgb2gray(I); % Convertir en niveaux de gris
    end

    for colonne = 1:hauteur
        for ligne = 1:largeur
            if(ecartType(colonne,ligne)>20)
                imageFiltree(colonne,ligne) = 255;
                if(abs(uint8(moyenne(colonne,ligne))-I(colonne,ligne))>20)
                    imageFiltree(colonne,ligne) = 0;

if(H(colonne,ligne)>0.4&&H(colonne,ligne)<0.90&&S(colonne,ligne)<0.5&&S(colonne,lig
ne)>0.1&&V(colonne,ligne)<0.4&&V(colonne,ligne)>0.05)
                        imageFiltree(colonne,ligne) = 255;
                    end

if(luminance(colonne,ligne)<70&&luminance(colonne,ligne)>23&&Cb(colonne,ligne)>122&
&Cb(colonne,ligne)<142&&Cr(colonne,ligne)>122&&Cr(colonne,ligne)<133)
                        imageFiltree(colonne,ligne) = 255;
                    end
                end
            else
                imageFiltree(colonne,ligne) = 0;
            end
        end
    end
```

```
%     if(compteurIteration == 1)
%         figure;
%         imshow(uint8(imageFiltree));
%     end
%
%     if(compteurIteration == 101)
%         figure;
%         imshow(uint8(imageFiltree));
%     end
%
%     if(compteurIteration == 201)
%         figure;
%         imshow(uint8(imageFiltree));
%     end
%
%     if(compteurIteration == 301)
%         figure;
%         imshow(uint8(imageFiltree));
%     end
%
%     if(compteurIteration == 401)
%         figure;
%         imshow(uint8(imageFiltree));
%     end
%
%     if(compteurIteration == 501)
%         figure;
%         imshow(uint8(imageFiltree));
%     end
%
%     if(compteurIteration == 601)
%         figure;
%         imshow(uint8(imageFiltree));
%     end

    %figure;
    %imshow(uint8(imageFiltree));
    compteurIteration = compteurIteration + 1;
end



%--------------------------------- Q2 ---------------------------------
compteurIteration = 1;
for compteurIteration = 1:nbIterations
    if(compteurIteration < 10)
        I = imread(['image_00' num2str(compteurIteration, '%01d') '.jpg']); %
Charger l'image
    end

    if(compteurIteration >= 10 && compteurIteration < 100)
        I = imread(['image_0' num2str(compteurIteration, '%02d') '.jpg']);
    end

    if(compteurIteration >= 100)
        I = imread(['image_' num2str(compteurIteration, '%02d') '.jpg']);
    end

    if ndims(I) == 3
        I = rgb2gray(I); % Convertir en niveaux de gris
    end

    for colonne = 1:hauteur
        for ligne = 1:largeur
            imageChargee(compteurIteration,colonne,ligne) = I(colonne,ligne);
        end
```

```
    end
end

tableauGaussSomme = zeros(hauteur,largeur);

for colonne = 1:hauteur
    for ligne = 1:largeur
        tableauPixel = zeros(1,nbIterations);
        tableauGauss = zeros(1,256);

        for compteurIteration = 1:nbIterations
            tableauPixel(compteurIteration) =
imageChargee(compteurIteration,colonne,ligne);
        end

%         figure;
%         plot(tableauPixel);
%         ylim([0, 255]);
%         xlim([0, nbIterations]);
%         xlabel('Nombre itérations')
%         ylabel('Valeur du pixel en niveau de gris')


        for cptIt = 1:nbIterations
            tableauGauss(tableauPixel(cptIt)+1) =
tableauGauss(tableauPixel(cptIt)+1)+1;
        end

%         tableauGaussSomme(colonne,ligne) = sum(tableauGauss);
%         figure;
%         plot(tableauGauss);
%         xlim([0, 256]);
%         xlabel('Niveau de gris')
%         ylabel('Occurence')

        maxTableau = max(tableauGauss);
        seuil = maxTableau*0.1;
        nbElementMoyenne = 0;
        moy = 0;

        for compteurSeuil = 1:256
            if(tableauGauss(compteurSeuil)>seuil)
                moy = moy + compteurSeuil*tableauGauss(compteurSeuil);
                nbElementMoyenne = nbElementMoyenne + tableauGauss(compteurSeuil);
            end
        end

        % Calcul 2eme moyenne
        moyenne(colonne,ligne) = (moy/nbElementMoyenne)-1;
        ligne;
        end

end

% Calcul 2eme ecart-type
for colonne = 1:hauteur
     for ligne = 1:largeur
        for nbImages = 1:nbIterations
             ecartType(colonne,ligne) =
ecartType(colonne,ligne)+(((imageChargee(nbImages,colonne,ligne)-
moyenne(colonne,ligne))^2)/nbIterations);
         end
         ecartType(colonne,ligne) = sqrt(ecartType(colonne,ligne));
     end
end
```

```
    figure;
    imshow(uint8(moyenne)); % Affichage de la moyenne en 1 image
    figure;
    imshow(uint8(ecartType)); % Affichage de l'écart type en 1 image
    %figure;
    %imshow(uint8(imageFiltree));
```